GITHUB AND DIGITAL REPOSITORY MANAGEMENT- FINAL PROJECT

IN-MEXICO PROGRAM BACKEND DEVELOPER CERTIFICATION

Valeria Anel Duque Salgado
16/09/2025
NAO ID: 3306

Contents:

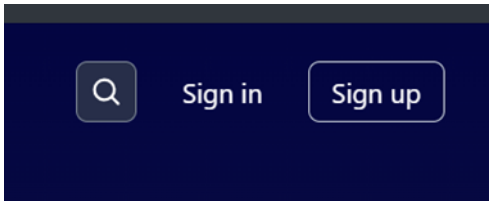
BACKLOG

User Story	Requirements
I want to create a new branch from the Master branch So that I can work on a separate set of changes without affecting the main codebase	Must be able to create a branch named "A" and another named "B" from Master. Branches should be visible in local and remote repositories. Branch creation should not affect Master.
As a developer, I want to create a docs.txt file in branch "A" and a different docs.txt in branch "B" So that I can add branch-specific documentation	The docs.txt file in branch A must be unique to that branch. The docs.txt file in branch B must be different from branch A. Files must be committed and pushed to their respective branches.
As a developer, I want to create a PR for branch "A" and branch "B" So that my changes can be reviewed before merging into Master	PRs must be created for both branches in GitHub. PRs must not be approved immediately. PRs should include a clear title and description.
As a developer, I want to resolve conflicts between branch A and branch B So that both branches can be merged safely into Master	Identify files causing conflicts. Merge changes correctly without losing data. Test changes after resolving conflicts to ensure integrity.
As a developer, I want to approve both PRs and merge them into Master So that all changes from branches A and B are incorporated into the main codebase	PRs must be approved in GitHub. Merges must not override important changes. Master branch must reflect all changes from both branches after merging.

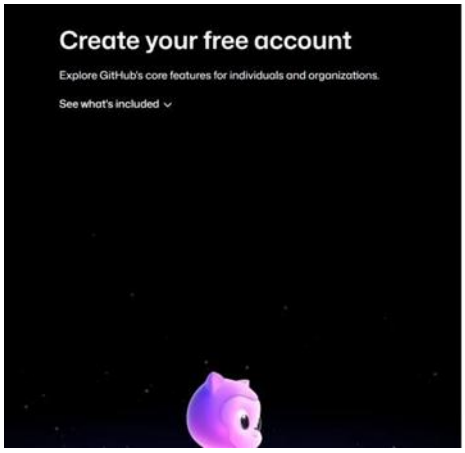
Requirements	Stages	Time estimation	Deliverables
Ability to create branches from Master. Branches A and B must exist in both local and remote repositories.	1. Checkout Master branch. 2. Create branch A (git checkout -b A). 3. Push branch A to remote (git push origin A). 4. Repeat steps for branch B.	15 minutes	Branch A and Branch B created and visible on GitHub.
Each branch must contain its own version of docs.txt. Files must be committed and pushed correctly.	1. Switch to branch A. 2. Create docs.txt with unique content. 3. Stage, commit, and push changes. 4. Switch to branch B and repeat with different content.	20 minutes	Unique docs.txt file in branch A. Different docs.txt file in branch B.
PRs must be created in GitHub for both branches. PRs must include a clear title and description. PRs should not be approved yet.	1. Open GitHub. 2. Create PR for branch A → Master. 3. Create PR for branch B → Master. 4. Add proper descriptions to both PRs.	15 minutes.	Two open PRs: A → Master, B → Master.
Detect merge conflict in docs.txt. Ensure no important content is lost. Push resolved version back to repository.	1. Attempt to merge PRs (GitHub flags conflicts). 2. Checkout conflicting branch locally. 3. Manually resolve docs.txt conflict. 4. Commit resolved file and push changes. 5. Verify in GitHub that conflicts are cleared.	30–40 minutes.	Conflict-free version of docs.txt. Updated branches ready to merge.

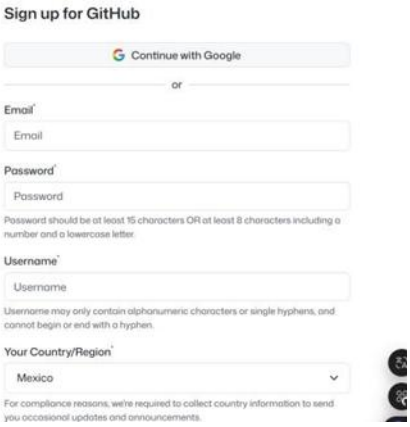
GITHUB ACCOUNT CREATION:

Go to https://github.com/ and click on Sign Up:

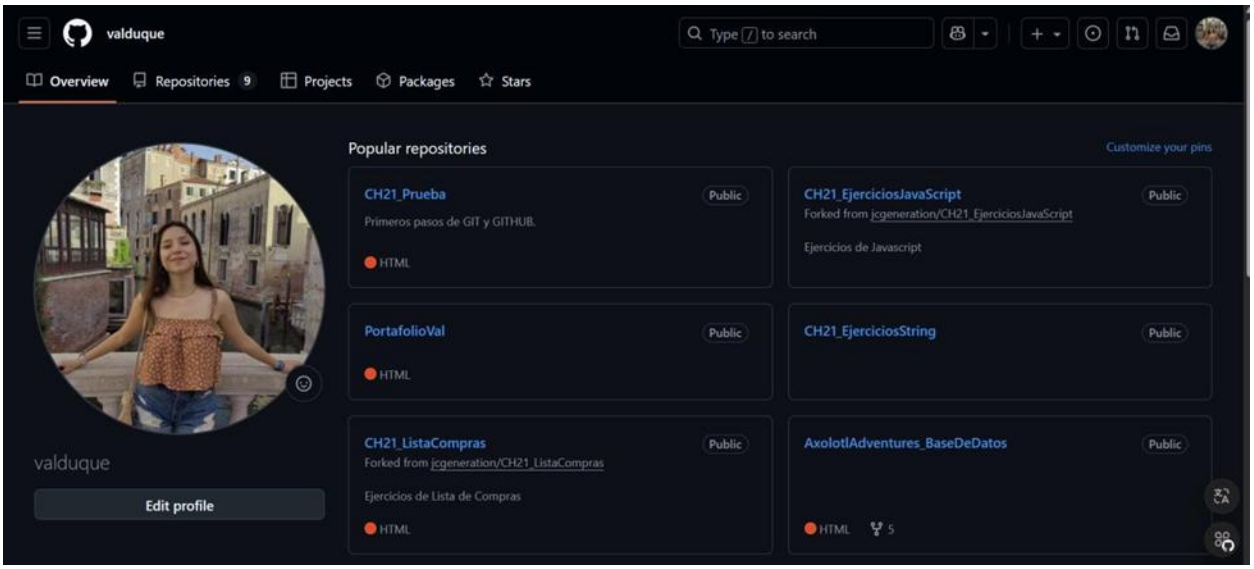


Then you can create an account through Google or with an email:





Once filled the information spaces and submitted the necessary information, the page will lead you to your new account:

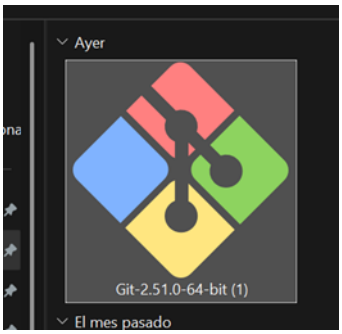


GIT INSTALLATION:

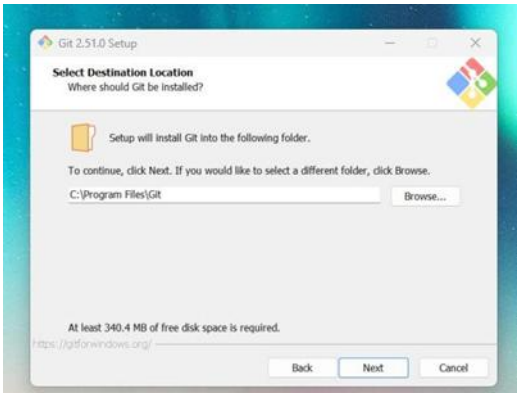
1. DOWNLOAD THE GIT SETUP FILES:

Go to the Git for Windows page (https://gitforwindows.org/) and download the setup files:

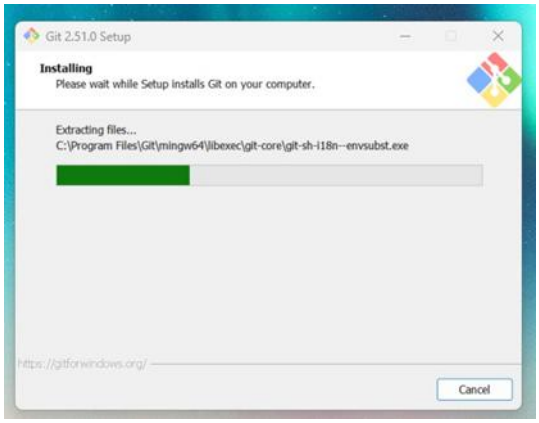
Once downloaded, start the installation and grant the corresponding permission:



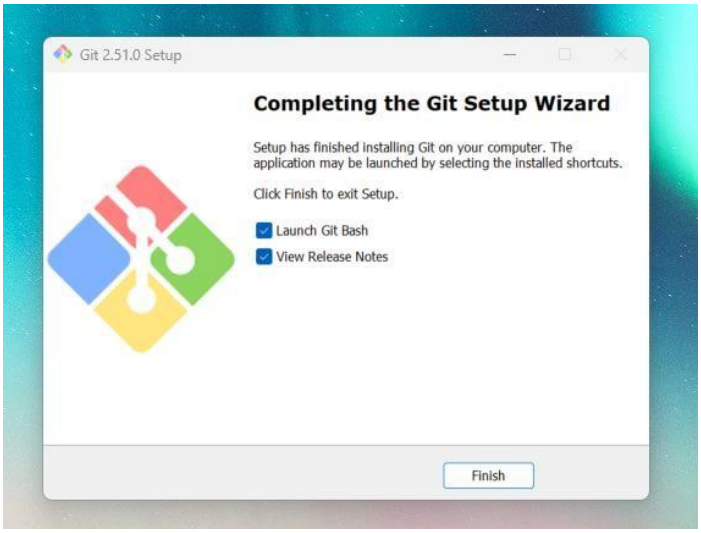
Start the installation system and select the Destination Location:



Select your installation preferences and let it install itself:

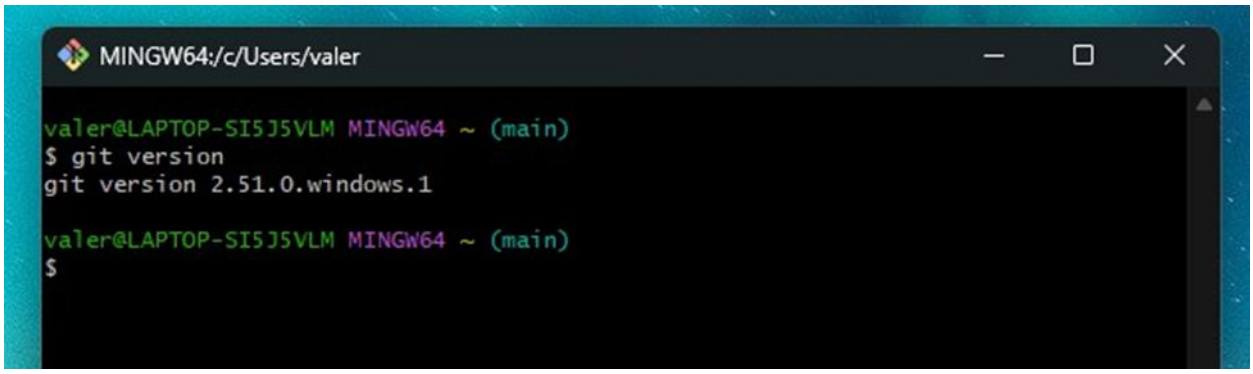


Finish the installation process:



2. VERIFICATION:

Open Git Bash and write down “git version” on the terminal to make sure everything is correct:



SSHKEYGENERATIONANDCONFIGURATION:

3. CREATION OF A SSH KEY ON GITBASH:

Open GitBash and execute the command ssh-keygen -t ed25519 -C your_email@example.com, and replace the email example with your own email:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~ (main)
$ ssh-keygen -t ed25519 -C "valeria.duques99@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/valer/.ssh/id_ed25519):
```

The system asks for a file to save the key, if you want to accept the default location of the key just press "enter":

```
Enter file in which to save the key (/c/Users/valer/.ssh/id_ed25519):
```

If it already exists, you can override it:

```
/c/Users/valer/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
```

Then it must be added a passphrase. And enter the same passphrase again for confirmation:

```
Enter passphrase for "/c/Users/valer/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
```

If everything is set up correctly, this is a message you shall receive:

```
Your identification has been saved in /c/Users/valer/.ssh/id_ed25519
Your public key has been saved in /c/Users/valer/.ssh/id_ed25519.pub
The key fingerprint is:
```

4. CONNECT GIT BASH WITH GITHUB:

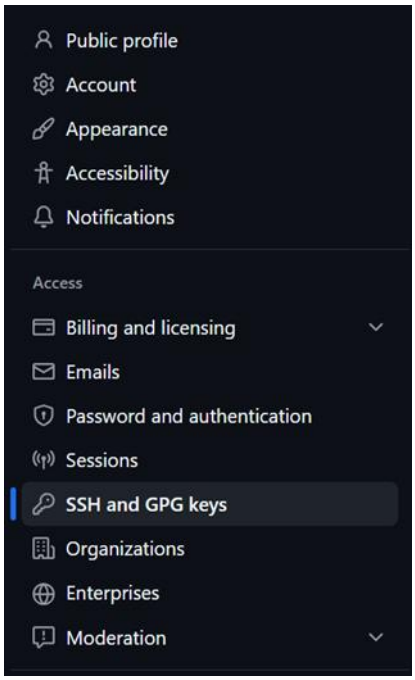
Write down on GitBash this command ls -al ~/.ssh, this is the outcome:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~ (main)
$ ls -al ~/.ssh
total 31
drwxr-xr-x 1 valer 197609   0 Dec  1  2022 ./
drwxr-xr-x 1 valer 197609   0 Sep  8 18:13 ../
-rw-r--r-- 1 valer 197609 464 Sep  9 10:13 id_ed25519
-rw-r--r-- 1 valer 197609 108 Sep  9 10:13 id_ed25519.pub
-rw-r--r-- 1 valer 197609 656 Dec  1  2022 known_hosts
-rw-r--r-- 1 valer 197609  92 Dec  1  2022 known_hosts.old
```

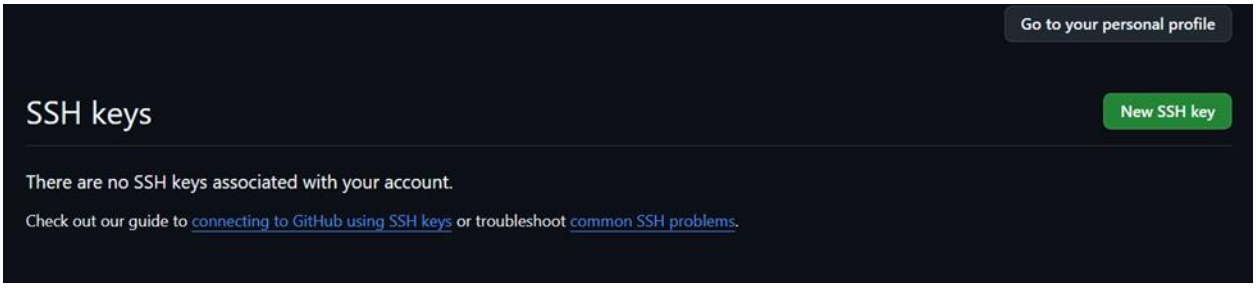
We need the one ending with .pub , then we write the command cat ~/.ssh/id_rsa.pub, or in this case; cat ~/.ssh/id_ed25519.pub.
This is the outcome:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~ (main)
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGX6Ihd8Z8F/Vc+5ROIEOdK6TaMBYrHOOpK53FZq
ltlI valeria.duques99@gmail.com
```

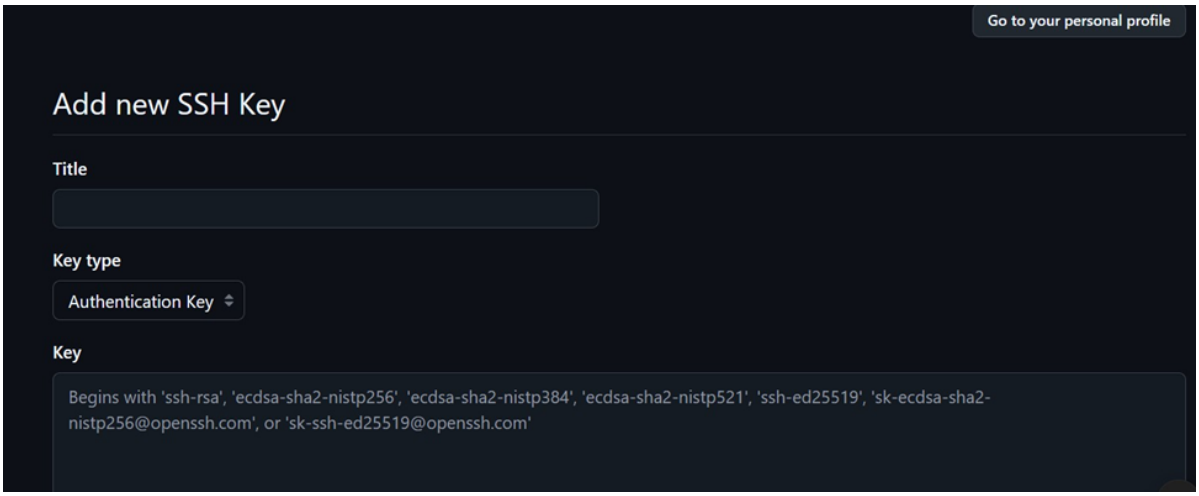
Full code is needed. It must be copied.
Then, we go to GitHub → Settings → SSH and GPG keys:



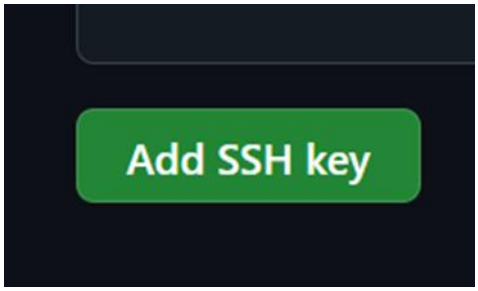
Click on "New SSH Key":



After that, a ssh key creation page opens. The information must be filled up.
The key created on GitBash and previously copied will be placed in the last section of the page:



Once everything is done, click on "Add SSH Key" button:



Remember that your code must include email as well. If everything is correct, this message will appear:

5. VERIFY ON GITBASH:

To verify the connection with the account, write on GitBash the next command: ssh -T git@github.com
GitBash Will ask you for your passphrase, previously created:

If it is connected, this message will appear:

```
$ ssh -T git@github.com
Enter passphrase for key '/c/Users/valer/.ssh/id_ed25519':
Hi valduque! You've successfully authenticated, but GitHub does not provide s
hell access.
```

Can also be checked, by adding the command the "git config --get user.email" onto GitBash:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~ (main)
$ git config --get user.email
valeria.duques99@gmail.com
```

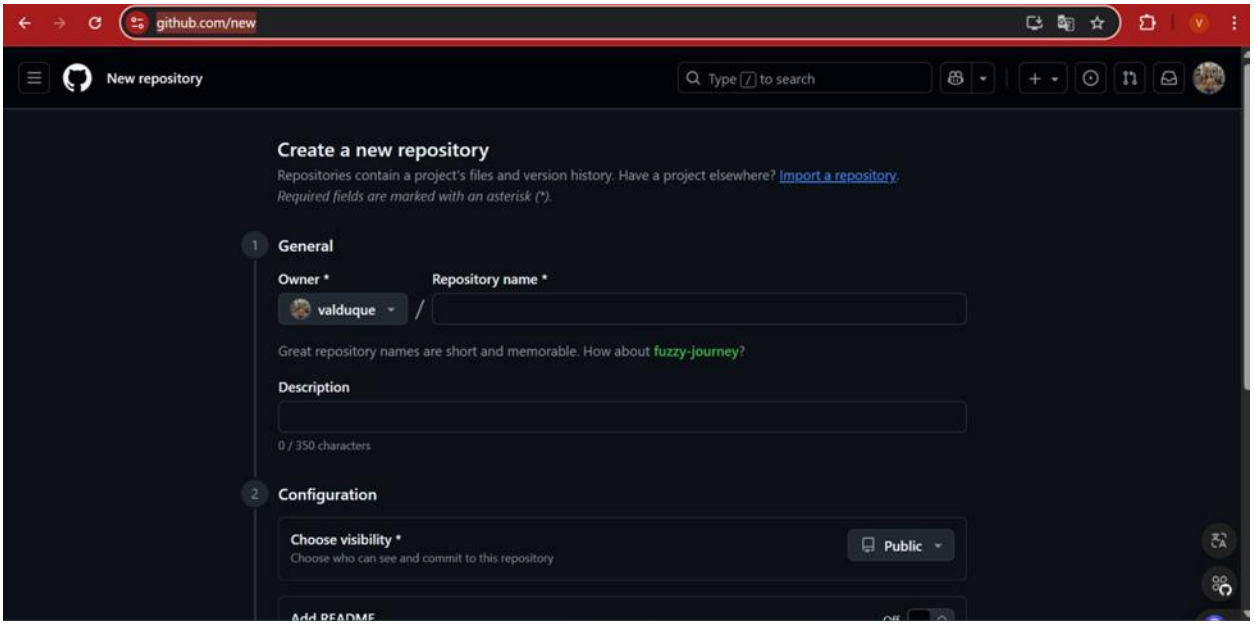
Or, can write the command "git config –list", which will bring the full list of the GitBash configuration:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~ (main)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
core.editor="C:\Users\valer\AppData\Local\Programs\Microsoft VS Code\bin\code
" --wait
init.defaultbranch=main
```

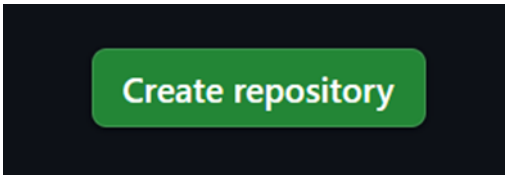
REPOSITORY CREATION IN GITHUB AND PROJECT UPLOAD:

1. Repository creation:

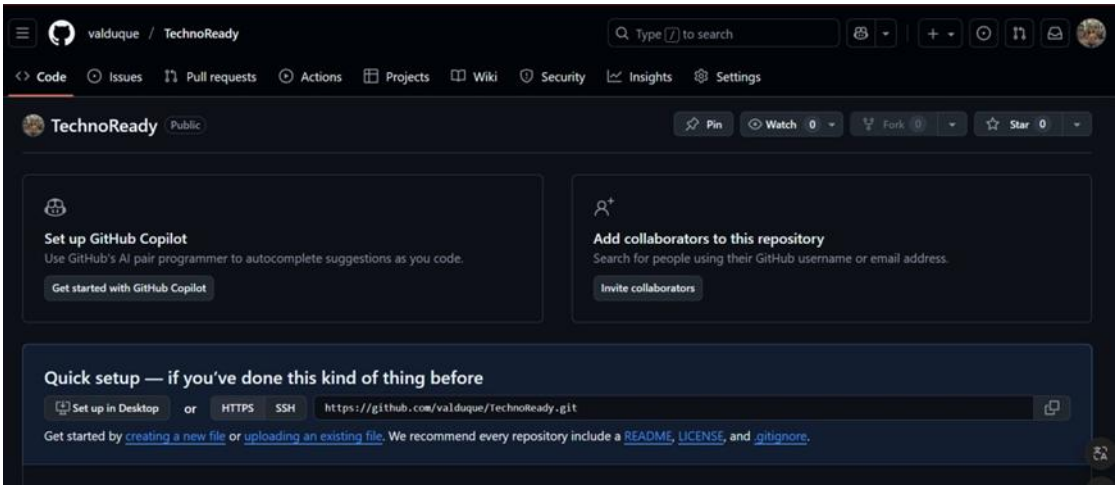
Inside the GitHub account, we must go to https://github.com/new . So, we can customize our repository:



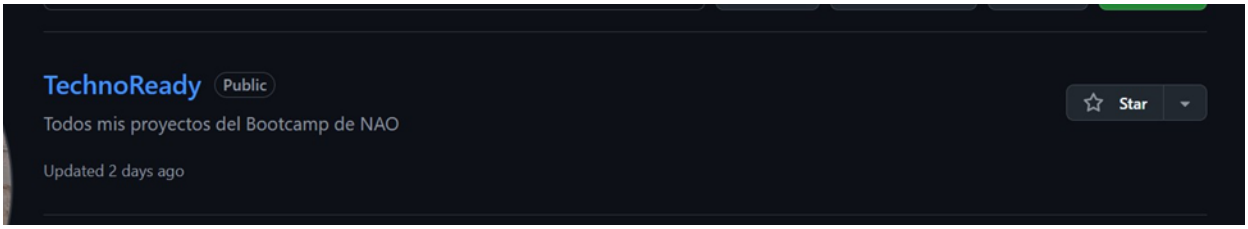
After that, just click the "Create repository" button:



And now you can see your repository's page:



Also, it'll appear on your repositories' list:



2. Repository connection with local repository:

From the local repository, GitBash must be open and initialize with the "git init" command:

Then add the command "git init add origin" followed by the URL of you GitHub repository:

PROJECT UPLOAD:

Inside our repository, and once our git is initiated, git execute de git add command, this will send all our files to the staging area:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git add .
```

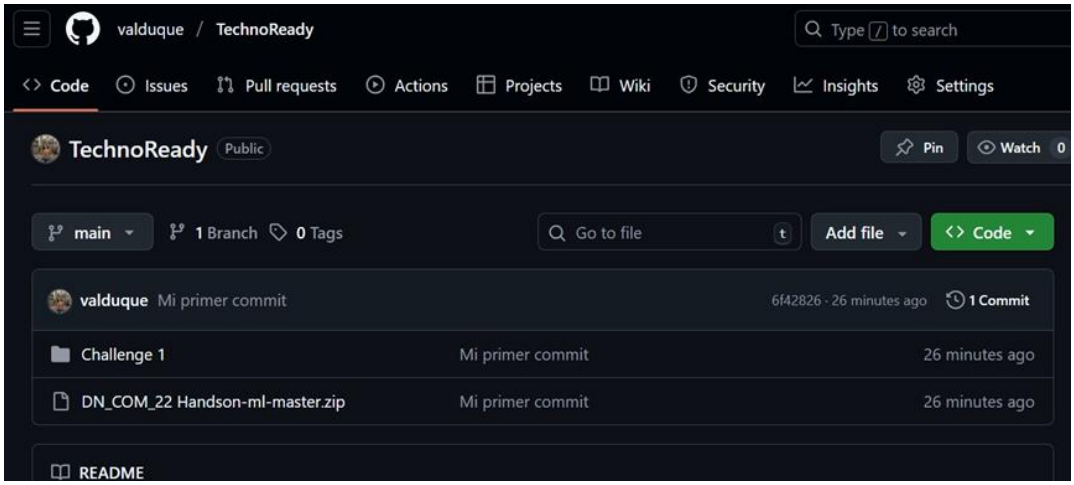
Then, we create a commit with the git commit -m “message” and press enter:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git commit -m "Mi primer commit"
[main (root-commit) 6f42826] Mi primer commit
 10 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Challenge 1/Develop.docx
 create mode 100644 Challenge 1/Plan.docx
 create mode 100644 Challenge 1/SPRINT 1/Develop.pdf
 create mode 100644 Challenge 1/SPRINT 1/Github link.pdf
 create mode 100644 Challenge 1/SPRINT 1/Plan.pdf
 create mode 100644 Challenge 1/SPRINT 1/ROADMAP.pdf
 create mode 100644 Challenge 1/SPRINT 2/Develop.docx
 create mode 100644 Challenge 1/SPRINT 2/~$evelop.docx
 create mode 100644 Challenge 1/SPRINT 2/~WRL0003.tmp
 create mode 100644 DN_COM_22 Handson-ml-master.zip
```

The just write the command git push origin branch.name:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 18.28 MiB | 3.33 MiB/s, done.
Total 14 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/valduque/TechnoReady
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

And now the project appears inside the repository on GitHub:



NEW BRANCH CREATION :

Inside your project, write the command git branch name.brach:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch develop
```

To confirm the branch creation, just write the command git branch:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch
  develop
* main
```

1. Update to the requirements.txt file:

If there is ever a desire to update something, there are some rules that must be followed. First, make the desired changes in your file:

```
MINGW64:/c/Users/valer/Documents/TechnoReady
  GNU nano 8.5                                    requirements.txt
Requirements list
User stories
needs
priorities
```

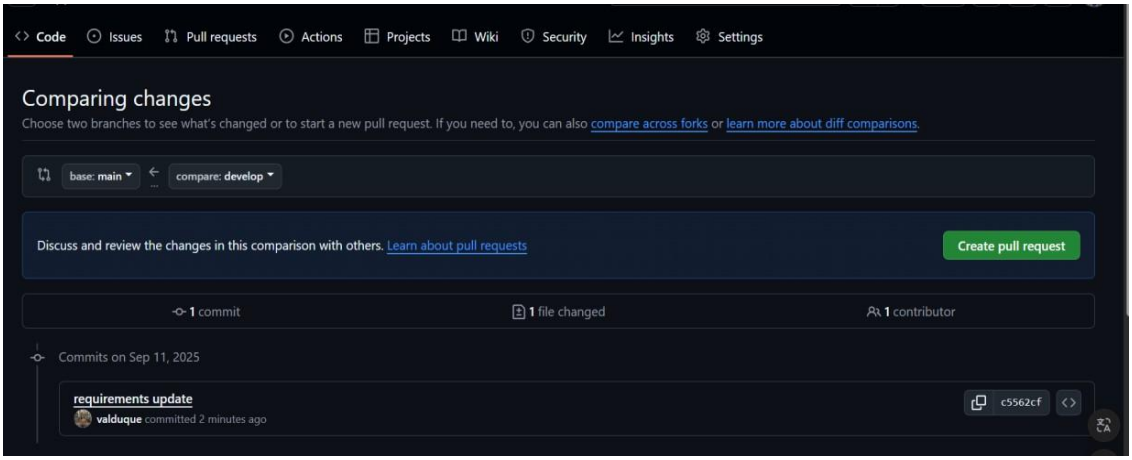
Then, save the changes with the combinations in the console below:

```
                                [ Read 4 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute     ^C Location   M-U Undo      M-A Set Mark    M-] To Bracket  M-B Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go To Line M-E Redo      M-6 Copy        ^B Where Was    M-F Next
```

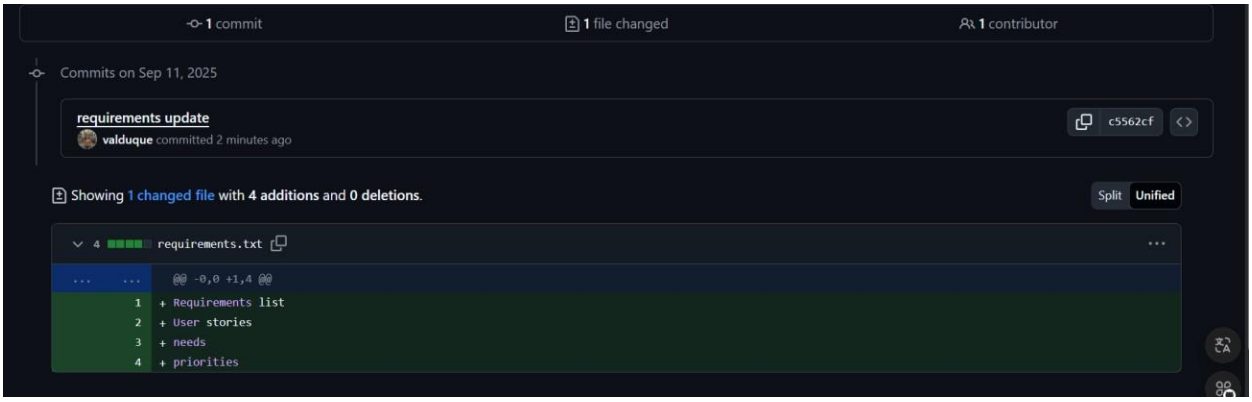
Once the changes have been done, you can upload them with a push to the desired branch.

CREATED PR :

If there is content uploaded that must be reviewed by other collaborators, it's possible to create a PR to make the whole team to be on the same page or at least, on the same code.
To do this, it must be already committed and pushed to the branch. Once it's done, in the project's menu should appear an option to create a pull request, it shall be clicked on the "create pull request" button:

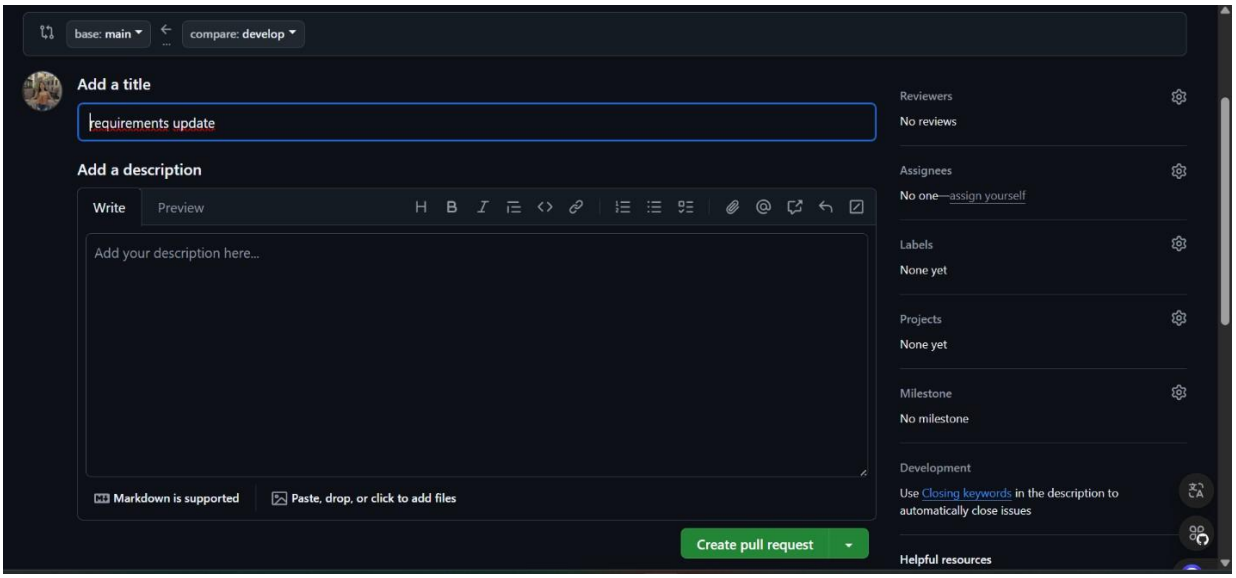


It will be able to show the changes:



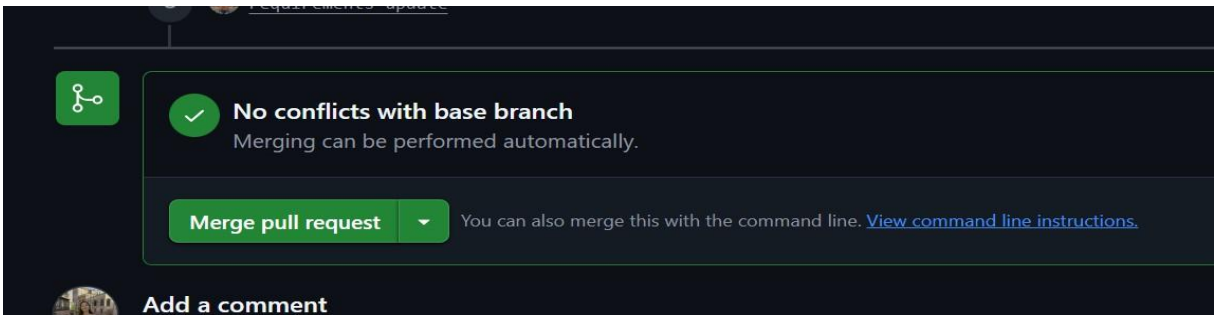
And now, it's possible to compare and do the pull request:

It can be added descriptions and comments:

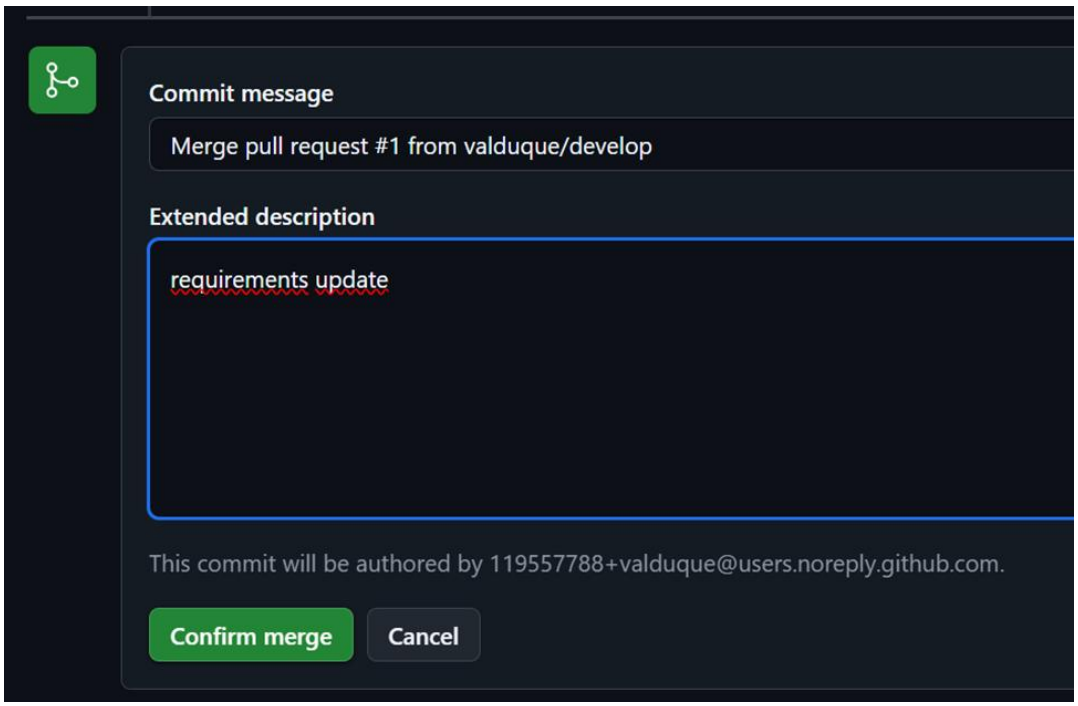


PR APPROVAL IN GITHUB:

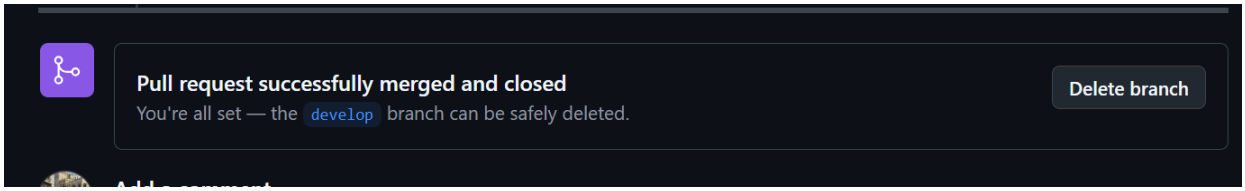
If a pull request is done on the project, a collaborator can approve another collaborator's work to be added to the project. The GitHub system will show if there is any conflict with the branch, or if it's not possible to merge with the new work. In this case, it's possible:



It must be clicked the "merge pull request" button and add any commentary if needed:



Lastly, the system will confirm that the action is finished successfully:



1. Creation of branch "A" from "Main":

From branch Main we just must write the command “git branch name.branch” in this case the branch will be called “A”:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch A
```

Then, switch to the new branch with the command “git checkout branch.name”:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git checkout A
Switched to branch 'A'
```

2. Creation of docs.txt in branch "A":

Inside the desired branch, write down the command “touch name_file.txt”:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ touch docs.txt
```

Add a few lines with the command “nano name_file.txt”:

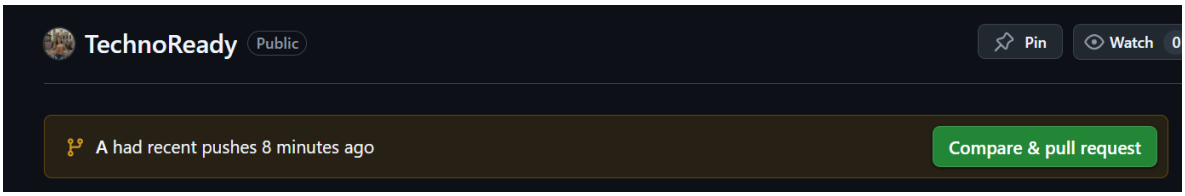
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ nano docs.txt
```

Add, commit and push all the changes with the command “git push -u origin A”:

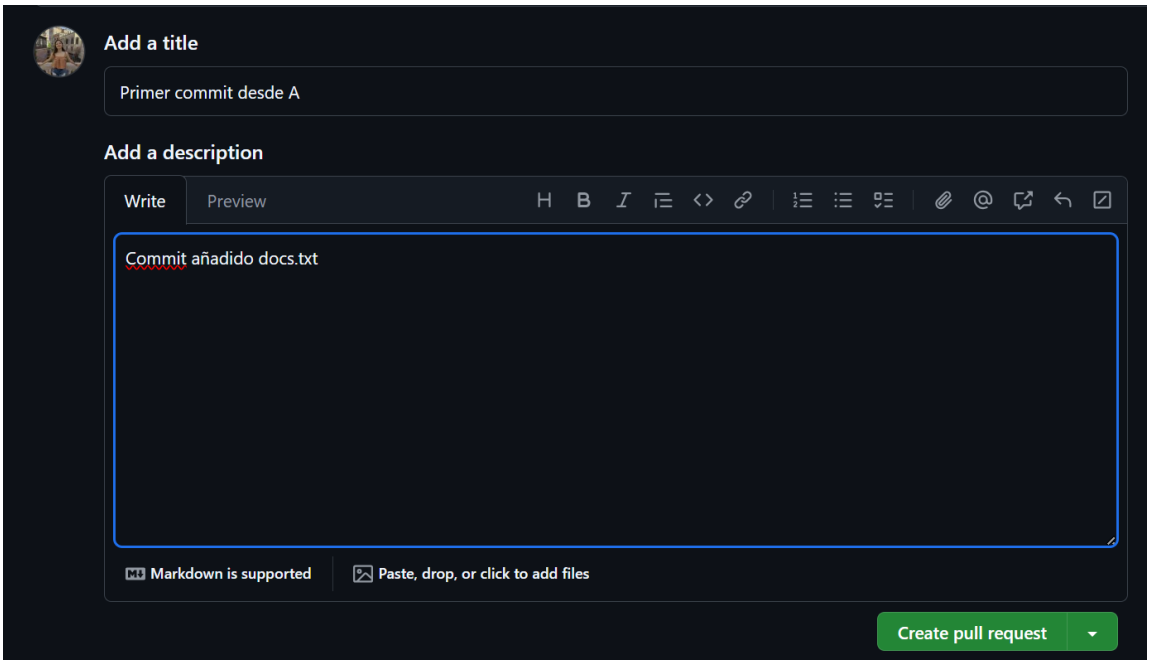
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ git push -u origin A
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 579 bytes | 289.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'A' on GitHub by visiting:
remote:      https://github.com/valduque/TechnoReady/pull/new/A
remote:
To https://github.com/valduque/TechnoReady
 * [new branch]      A -> A
branch 'A' set up to track 'origin/A'.
```

3. PR created for branch "A":

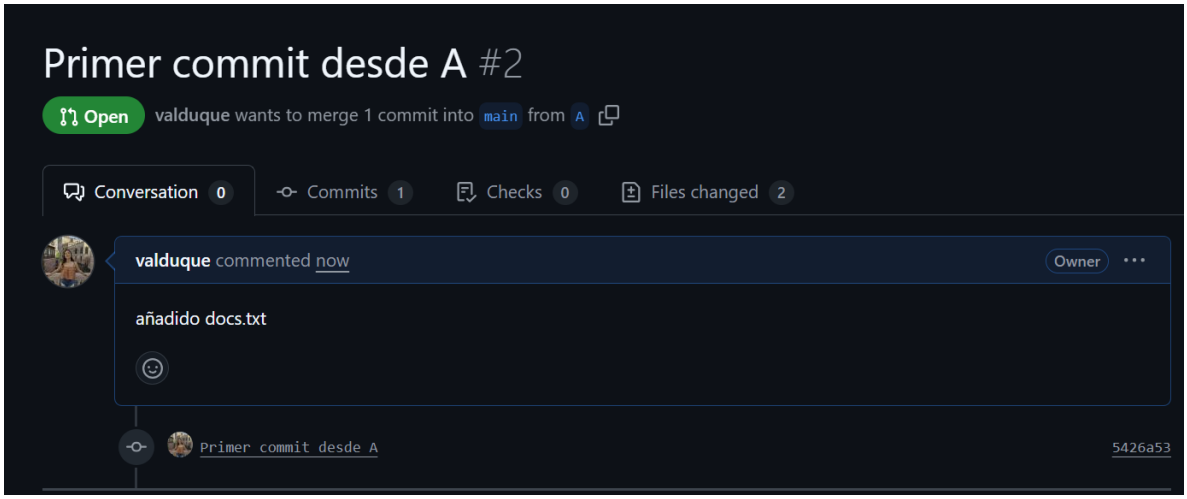
On the project's Github repository page, it will show the new branch and pushes created by it. "Comapre and pull request" will be clicked:



Then, the customizable page for the pull request will appear:



Once finished all the details, just press “create pull request”:



Creation of branch "B" from "Main":

From branch Main we just must write the command “git branch name.branch” in this case the branch will be called “B”:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch B
```

Then, switch to the new branch with the command “git checkout branch.name”:

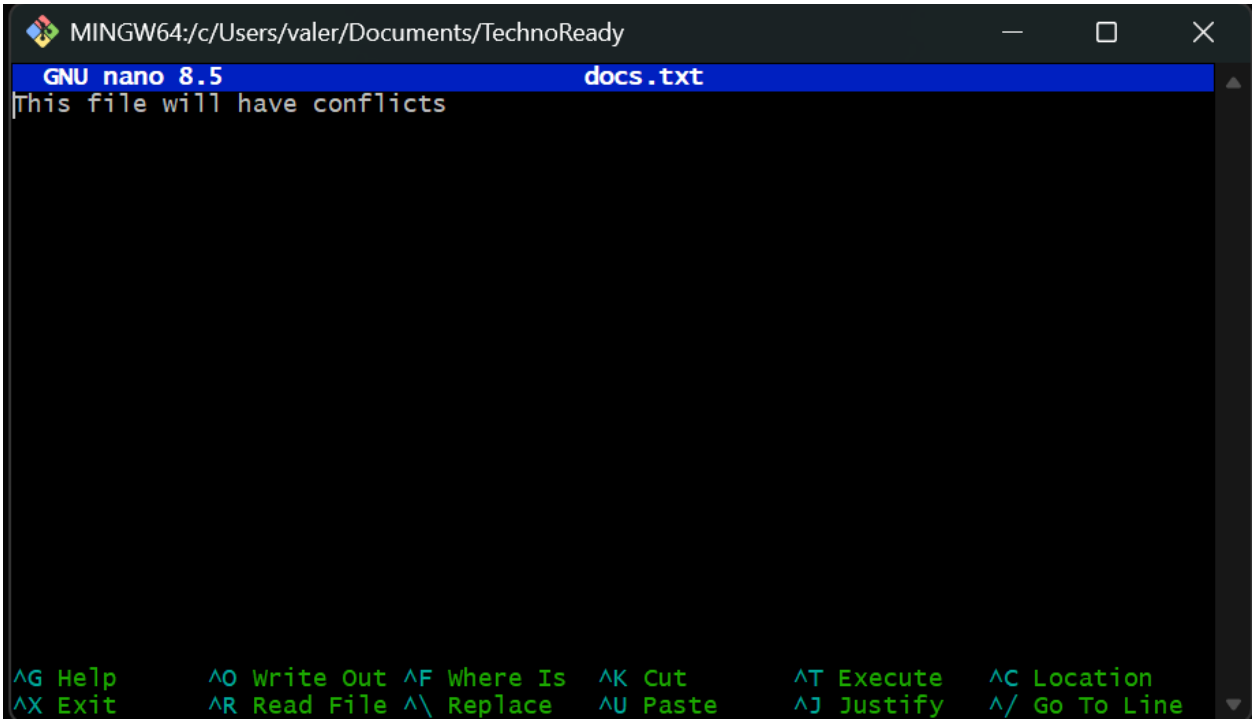
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git checkout B
Switched to branch 'B'
```

4. Creation of a different docs.txt in branch "B":

Inside the desired branch, write down the command “touch name_file.txt”:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ touch docs.txt
```

Add a few lines with the command “nano name_file.txt”:



Add, commit and push all the changes with the command "git push -u origin A":

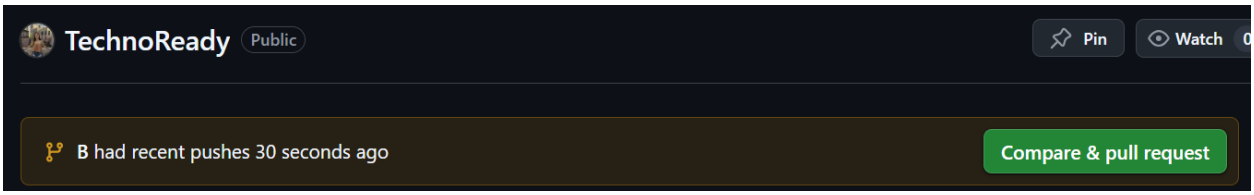
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git add .
warning: in the working copy of 'docs.txt', LF will be replaced by CRLF the next
 time Git touches it
```

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git commit -m "Primer commit en B"
[B a91ba89] Primer commit en B
 1 file changed, 1 insertion(+)
 create mode 100644 docs.txt
```

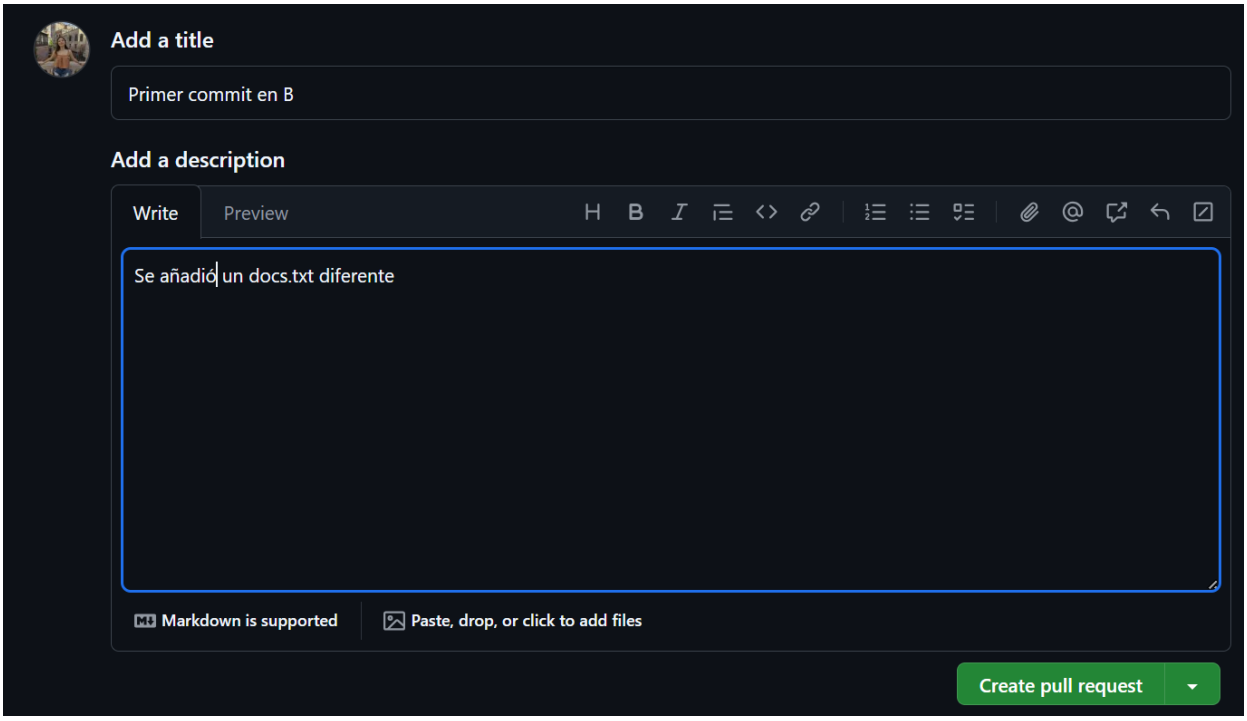
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git push -u origin B
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 301 bytes | 150.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'B' on GitHub by visiting:
remote:      https://github.com/valduque/TechnoReady/pull/new/B
remote:
To https://github.com/valduque/TechnoReady
 * [new branch]      B -> B
branch 'B' set up to track 'origin/B'.
```

5. PR created for branch "B":

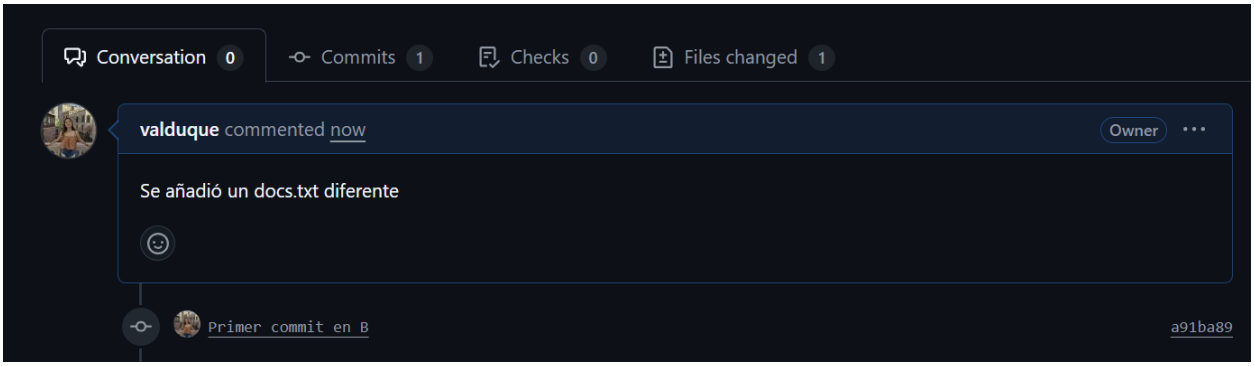
On the project's Github repository page, it will show the new branch and pushes created by it. "Comapre and pull request" will be clicked:



Then, the customizable page for the pull request will appear:



Once finished all the details, just press "create pull request":



CONFLICT BETWEEN BRANCHES:

When we want to merge both branches, the creation of docs.txt, will bring up problems. First, let's go to the main branch:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git checkout main
M       Challenge 1/Develop.docx
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

In main branch, let's merge with the A branch with main branch:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge A
Updating 16c1a45..5426a53
Fast-forward
 Challenge 1/~$evelop.docx | Bin 0 -> 162 bytes
 docs.txt                  |   1 +
 2 files changed, 1 insertion(+)
 create mode 100644 Challenge 1/~$evelop.docx
 create mode 100644 docs.txt
```

Then, merge the B branch with the main, to get the conflict:

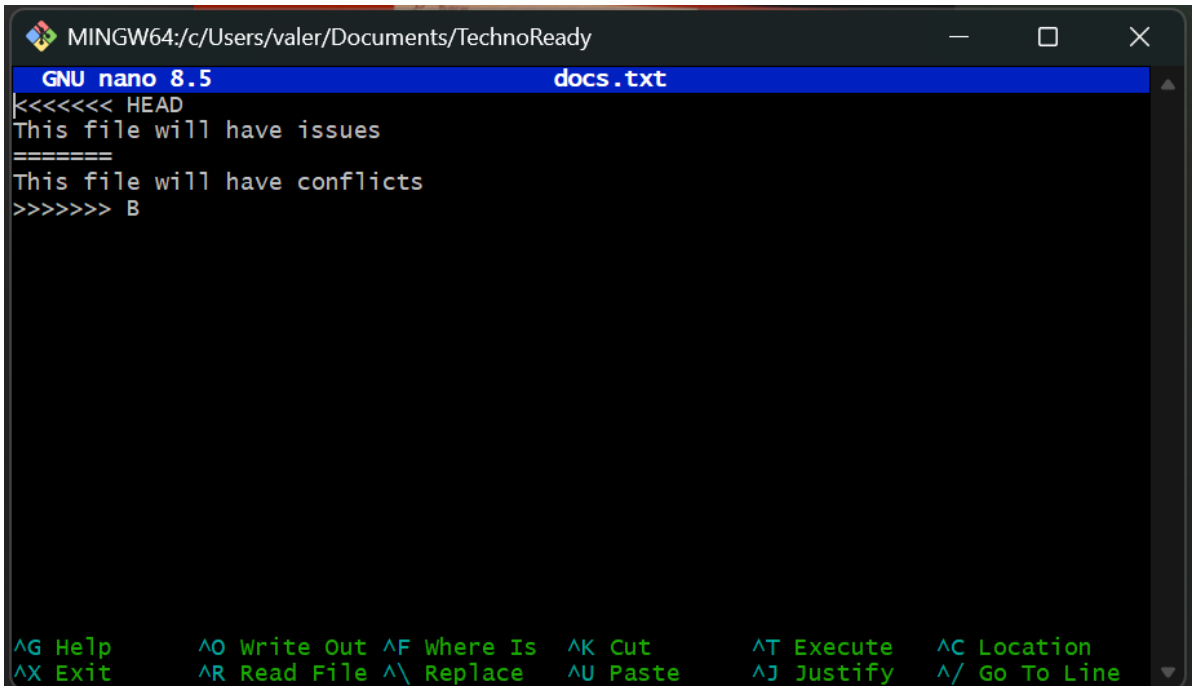
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge B
Auto-merging docs.txt
CONFLICT (add/add): Merge conflict in docs.txt
Automatic merge failed; fix conflicts and then commit the result.
```

When this message It's showed, that means we are having a conflict between branches:

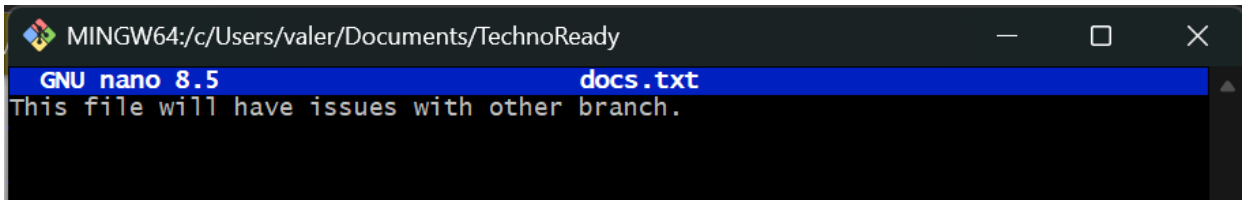
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge B
Auto-merging docs.txt
CONFLICT (add/add): Merge conflict in docs.txt
Automatic merge failed; fix conflicts and then commit the result.
```

CONFLICT RESOLUTION PROCESS:

We face a challenge with the docs.txt, since now it has conflict:

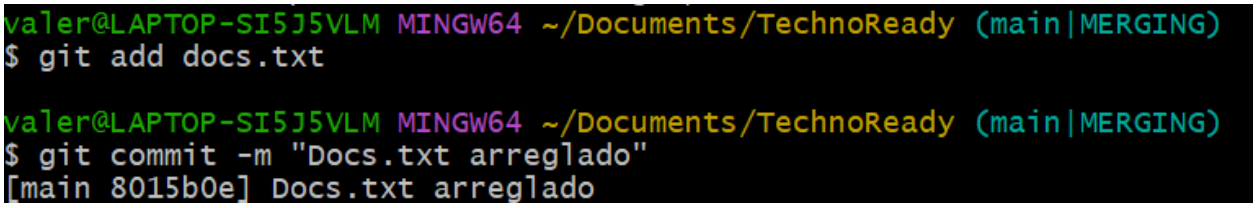


For this exercise, it was chosen to make the changes manually:



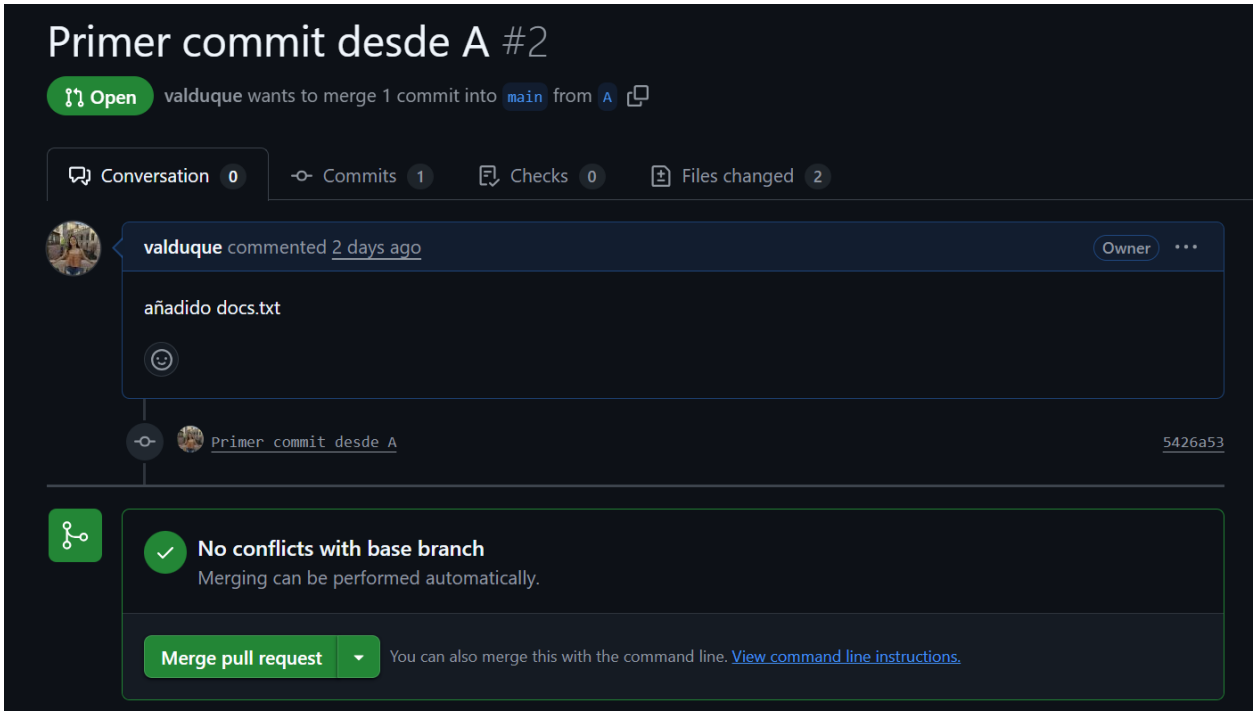
GIT COMMAND USED TO REVERT CHANGES:

Once the changes are made. We finish the merge:

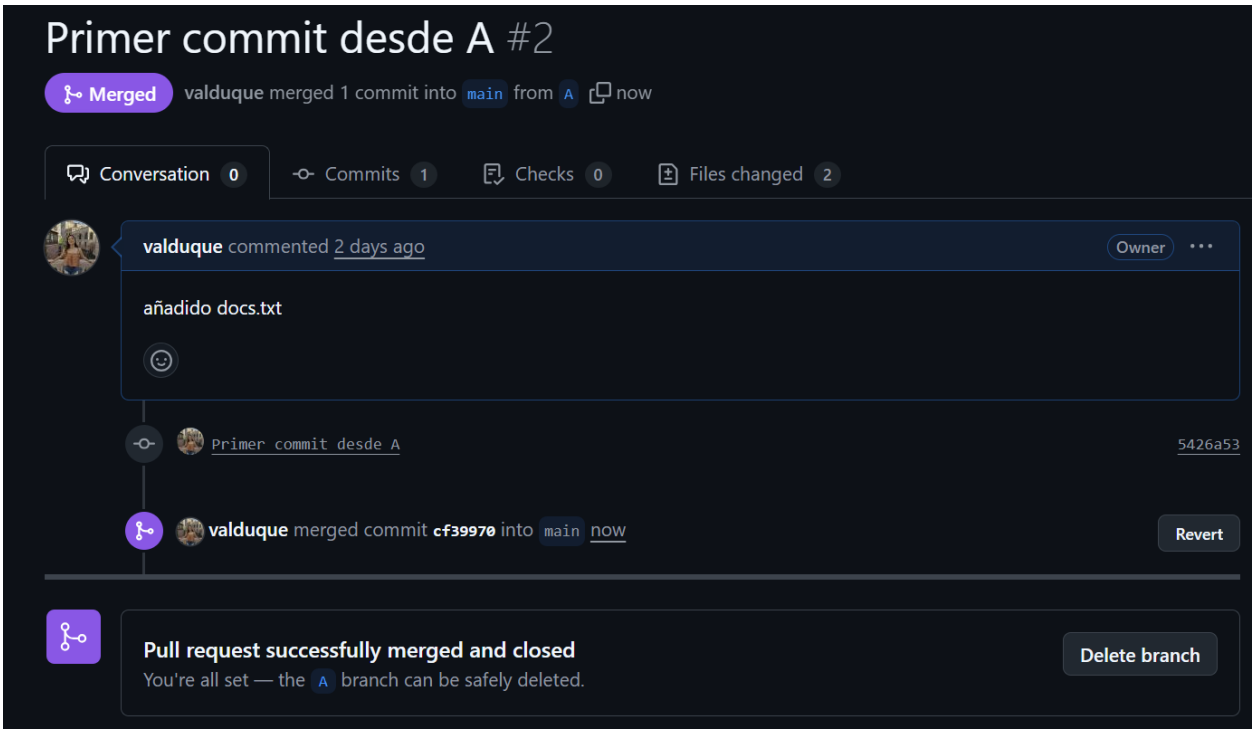


APPROVED PRS AND MERGED BRANCHES:

Now, we can approve the pull request for both branches, in GitHub:

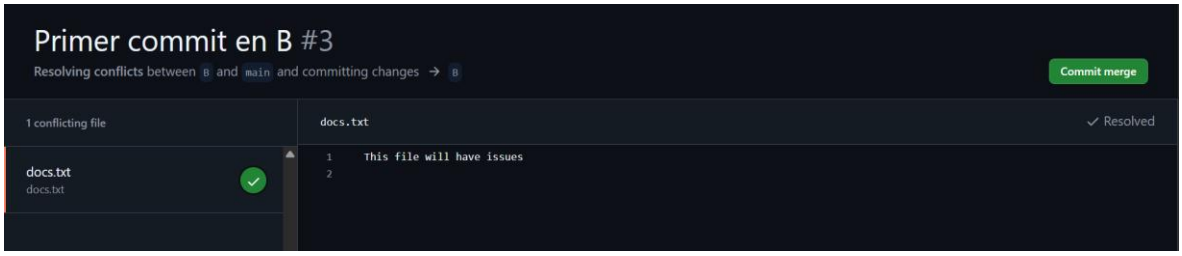


And everything is correct and working fine:

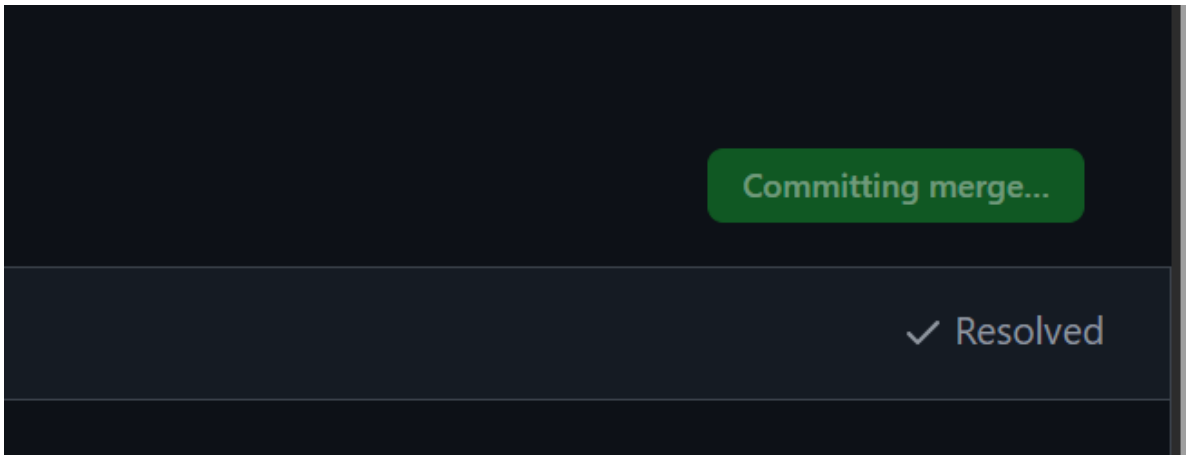


6. Pull request the other branch:

The other branch may or may not require you to fix the conflicted file. If you have to solve it just do it manually again on GitHub:



Then commit the merge:



And if there is not any issue, merge again: